IoT Discovery Protocols: a MQTT-D and UPnP Comparison in Dense Network

Daniele Foschi
Alma Mater Studiorum Università di Bologna UNIBO
Bologna, Italy
daniele.foschi2@studio.unibo.it
https://github.com/DaniDF/MQTT-Discovery-vs-UPnP

***Abstract*—This paper provides a comparative evaluation of Universal Plug and Play (UPnP) and Message Queuing Telemetry Transport-Discovery (MQTT-D) based on two fundamental performance metrics: latency and robustness. The analysis examines the behavior of both protocols under similar deployment conditions, with particular attention to dense network environments, where numerous devices simultaneously attempt to discover one another. The experimental findings indicate that UPnP consistently surpasses MQTT-D in both aspects, demonstrating lower latency and higher robustness across all tests. These results suggest that UPnP may represent a more effective solution for dense application scenarios in which rapid response times are essential.**

***Index Terms*—discovery, mqtt-d, upnp.**

I. Introduction

THE concept of Internet of Things (IoT) has been described through a variety of definitions in the literature. The International Telecommunication Union is *"A global infrastructure for the information society, enabling advanced services by interconnecting (physical and virtual) things based on existing and evolving interoperable information and communication technologies"* [1]. In a complementary manner, IEEE identifies a set of essential characteristics that distinguish IoT. Specifically, it is viewed as a networked infrastructure in which objects are uniquely identifiable, communicate through the Internet in at least part of their communication process, and are capable of sensing, actuating, or both. Furthermore, such objects are expected to be programmable through the integration of intelligence, while the services they provide should be accessible at any time, in any place, and for any entity.

In a environment populated by a large variety of different devices with different needs and capabilities designing a standard useful for communicating with them is a challenge. Discovery protocols are designed to reconcile the dichotomy between the requirement for highly specific, ad-hoc protocols capable of granular device control and the necessity for generalised frameworks that can accommodate a highly heterogeneous ecosystem of devices. By employing these protocols, an IoT device achieves autonomous configuration through active participation in the advertisement and solicitation of network-based services. Devices functioning as service providers systematically broadcast their available resources, the specific services they offer, and the operational guidelines for invocation, such as the appropriate Application Programming Interfaces (APIs). Furthermore, discovery protocols streamline the connectivity paradigm between clients and target resources or services. Consequently, this architecture ensures that clients are adequately equipped to transmit necessary input parameters to the services and successfully retrieve the corresponding computational results.

In particular this dissertation is focused on Universal Plug and Play (UPnP) and a protocol based on Message Queuing Telemetry Transport (MQTT) knwon as MQTT-Discovery (MQTT-D). UPnP is a discovery protocol without infrastructural nodes where each node can advertise and listen for services. In particular nodes that make available services are called *devices* and those searching for services are called *control-points*. In MQTT-D there is a third entity called *broker* that facilitates the communication between services and clients.

This dissertation wants to highlight the performance differences between UPnP and MQTT-D in IoT environment. The main focus is measuring and comparing latency and robustness in different deployment configurations: how different amount of data producers and consumers affect the protocol performances. Section II provides an overview of IoT protocol categorisation, followed by a detailed description of the two protocols under consideration. Section III outlines the requirements for the experimental evaluation and describes the methodology adopted for conducting the tests. Section IV presents and discusses the obtained results, while Section V summarises the main findings and concludes this dissertation.

II. Discovery Protocols for IoT

This section discuss about IoT discovery protocols with particular focus on MQTT-D and UPnP.

IoT environments are characterise by a large variety of different devices therefore there must be mechanisms available for automatic discovery of resources, their properties and capabilities as well as the means to access them [2]. In [3] are presented different categorizations for IoT discovery technologies, this dissertation will be focused only on *"Searching on My Network"*. A further distinction can be made based on the topology that these protocols implement, in particular they can be categorised into: centralised and distributed. In

centralised approaches, resources are registered in a directory that serves as intermediary between providers and users. In distributed approaches, users discover active resources through direct communication with resources in a Peer-to-Peer (P2P) fashion with no need for an intermediary directory [4].

UPnP is a P2P distributed discovery service without hierarchical structuring. In UPnP two entities can be distinguished: *control-point* the entity that look for services and resources and *device* the entity that make services available. A *device* can dynamically join a network and obtain a locally valid *IP* address. After it can make its capabilities visible and discover others, if present, and dynamically leave the network. UPnP uses *XML* for the description of the resources, services and action invocations.

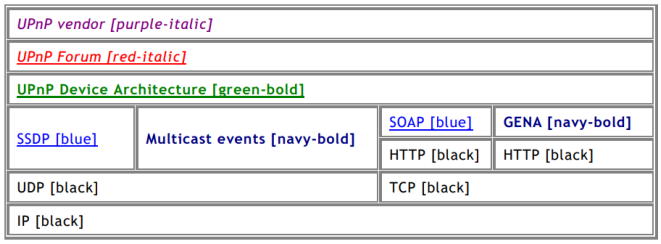


Fig. 1. UPnP Protocol Stack [5]

The UPnP protocol stack is based on four parts: Simple Service Discovery Protocol (SSDP) [5], HyperText Transfer Protocol (HTTP) [6], Simple Object Access Protocol (SOAP) [5] and General Event Notification Architecture (GENA) [5]. SSDP is a network protocol based on the Internet protocol suite for advertisement and discovery of network services and presence information. It is a text-based protocol based on a non fully compliant version of HTTP over UDP [5].

In SSDP, whenever a node intends to discover a *device*, it issues an M-SEARCH request specifying the desired device characteristics. Upon receiving this request, a *device* determines whether it satisfies the specified criteria and, if so, returns an appropriate response. Since the number of potential responses cannot be known in advance, a burst of simultaneous replies on the network may lead to overloading of the *control point*. To mitigate this risk, the *control point* includes the MX parameter in the search request [5]. This parameter defines the maximum number of seconds within which responding *devices* may transmit their replies. Consequently, each *device* waits for a random interval between 0 and the value specified by MX before sending its response.

SOAP is a messaging protocol specification for exchanging structured information in XML message format. It is used by the *control-point* for: (i) requesting the execution of services, Remote Procedure Call (RPC), with specified arguments, (ii) on the *device* and by the *device* for returning the results. GENA is an event notification protocol used by *devices* to publish variables updates to subscribed *control-points*. GENA supports both unicast and multicast event notification.

MQTT-D is a centralised discovery protocol MQTT-based [7]. MQTT is a lightweight publish-subscribe messaging protocol designed for constrained devices. MQTT-D is a metadata-exchange convention built on top of the MQTT protocol that standardises how devices and services describe themselves and publish their configuration to an MQTT broker. It enables automatic detection, registration, and integration of devices in home-automation or IoT ecosystems without requiring manual configuration.

The MQTT protocol assigns a specific Quality Of Service (QOS) level to each message, establishing the delivery guarantees required across potentially unreliable networks. The protocol defines three hierarchical levels of service: QOS 0 (At-Most-Once), which provides best-effort delivery without retransmission mechanisms; QOS 1 (At-Least-Once), which ensures delivery but permits the possibility of duplicate messages; and QOS 2 (Exactly-Once), the most complex level, which employs a multi-step handshake to guarantee that a message is delivered once and only once. Furthermore, publishers can append a *retained* flag to a message. This instructs the MQTT broker to persist the last message for a given topic, ensuring that any newly subscribed clients immediately receive the most recent data upon connection, independent of the initial publication event.

As MQTT-D does not define a standardised role-based classification for clients, this dissertation adopts, for the sake of terminological clarity, the same naming convention used in UPnP. Accordingly, entities that provide services are referred to as *devices*, whereas entities that discover and make use of such services are referred to as *control-points*.

	UPnP	**MQTT-D**
Architecture	Decentralized (P2P)	Centralized (Client-Broker)
Transport Protocol	UDP Multicast (HTTP over UDP)	TCP/IP
Network Scope	Local (LAN, Time-to-Live = 2 [5])	Global (not limited)
Data Format	XML	JSON

III. IoT Discovery Protocols Analysis

This dissertation aims to evaluate the performance of two of the main IoT discovery protocols, MQTT-D and UPnP. Such an evaluation is particularly relevant in dense IoT scenarios, including smart factory floors, smart cities, and extensive agricultural sensor grids, where system reliability and rapid device discovery are critical to maintaining overall network performance. For this reason the evaluated metrics for this study are latency and robustness. The first wants to measure the time needed for the execution of the first interaction between *control-point* and *device* after the discovery phase. The second wants to measure the ratio between the discovered devices and the actual deployed devices.

A. Implementation

To establish a testing environment, a fully compatible UPnP *device* and *control point* were implemented in accordance with

the official specification [5]. All major protocol components were developed, including SSDP, SOAP, and GENA [8]. For HTTP, the standard library provided by Go was employed. To further validate standards compliance and interoperability, a third-party UPnP client library was also used [9].

For MQTT-D, an existing implementation of the MQTT protocol was adopted, while only a wrapper for the MQTT-D data format was developed. As the MQTT broker, Mosquitto was selected due to its widespread adoption and popularity in IoT deployments.

MQTT-D defines two mechanisms for *device* presence discovery. The first relies on a retained message published by a *device* when it joins the network; this message is subsequently delivered by the broker to a *control point* when the latter subscribes to the corresponding topic [10]. The second mechanism is based on solicitation: the *control-point* publishes a message on a predetermined topic to which all *devices* are subscribed. Upon receiving such a solicitation message, each *device* responds by sending its discovery message [10].

The software developed for this work [8] supports both mechanisms; however, the experiments were conducted exclusively using the solicitation-based approach. All software components developed by the author [8] were implemented in Go in order to balance high execution efficiency with the advantages of a higher-level programming language.

IV. Experimental Results

This section evaluates latency and robustness in a two-node UPnP deployment, in which one node runs the *devices* and the other runs the *control-points*. The MQTT-D deployment requires the addition of a third node, which hosts the MQTT broker. Each node is equipped with 12 core CPU, 16GB of memory and 2.5Gb lan interface.

Experimental evaluations were executed in a closed, dedicated network infrastructure to isolate the testing equipment. All experiments were performed over both wired and wireless (IEEE 802.11n [11]) connections. The deployment of *devices* and *control-points* was scaled incrementally: initially in unit steps from 1 to 10, and subsequently in decadal steps up to 100. For MQTT, each configuration was evaluated across QOS levels 0, 1, and 2. To mitigate statistical variance, each experimental combination was repeated 30 times, and the results were averaged.

The experiments tried to evaluate the effectiveness of the MX parameter in the UPnP protocol by removing the random wait time from the *device* response. The following subsections present the measured metrics for configurations of *devices* and *control-points* of value 1, 5, 10, 50, 100 but a more detailed version can be found in appendix A.

A. Latency comparison

Figure 2 compares latency of UPnP with different *devices* and *control-points*. It reveals a clearly distinguishable performance difference between the configuration with a single deployed *device* and those with higher numbers of deployed *devices*. Furthermore, Figure 11a in Appendix A shows that the difference between the configurations with 2 and 3 deployed

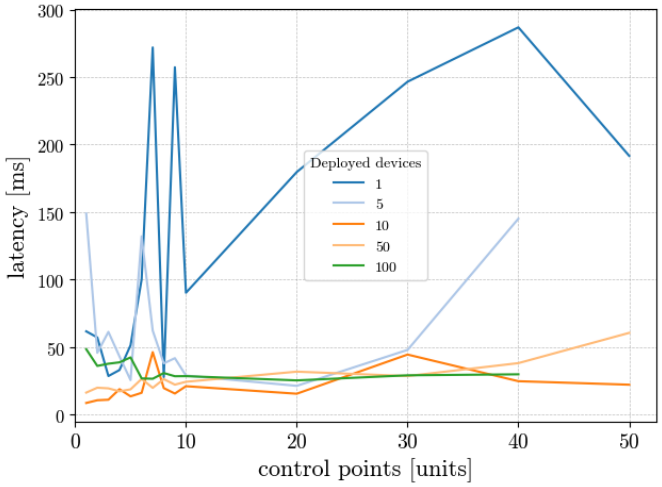


Fig. 2. UPnP latency comparison with different *devices* and *control-points*. See 11.

devices further supports the observation that setups with fewer deployed *devices* are associated with higher latency. By contrast, configurations involving more than 10 devices yield relatively stable latency values, which begin to diverge noticeably only when the number of *control points* exceeds 30.

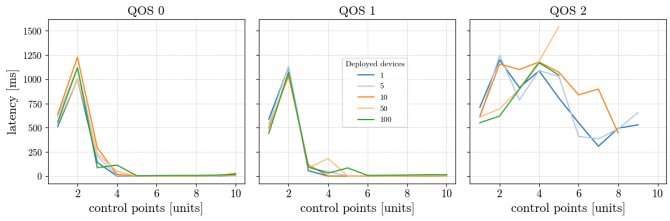


Fig. 3. MQTT latency comparison with different *devices control-points* and QOS. See 9a.

The Figure 3 compares latency of MQTT with different *devices* and *control-points*. The latency measurements obtained from the MQTT-D tests indicate a clear distinction between the lower QOS levels, QOS 0 and 1, and the highest level, QOS 2. As shown in Figure 3, configurations with the minimum number of deployed *control-points* exhibit a latency of approximately 600 ms. This value rises to around 1200 ms in the case of 2 deployed *control-points*, after which a sharp decrease is observed. By contrast, this behaviour is not evident in Figure 3, where the latency values corresponding to 1 and 2 deployed *control-points* remain similar, and the subsequent reduction does not occur at the same configuration point. Instead, elevated latency values persist up to at least 5 deployed *control points*.

1) Results comparison: The comparison between Figure 2 and Figure 3 indicates that UPnP consistently exhibits lower latency than MQTT-D. For configurations with more than one deployed *control point*, the observed latency values are approximately 8 times lower than those of the best-performing MQTT-D configuration.

B. Robustness comparison

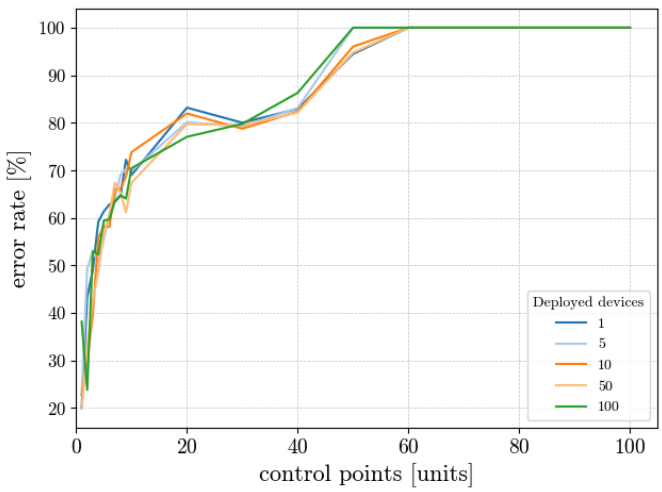


Fig. 4. UPnP robustness comparison with different *devices* and *control-points*. See 13.

In figure 4 is shown a comparison of the error rate in UPnP with different *devices* and *control-points*. For UPnP, the ratio between the number of *devices* discovered by the *control points* and the actual number of deployed *devices* is observed to be consistent across the different configurations (Figure 4). The error rate increases rapidly, following an approximately logarithmic trend up to 20 deployed *control points*; it then remains almost constant until 40 *control points*, after which it rises sharply toward 100%. A more detailed representation in Figure 13 further confirms that this behaviour is nearly identical for all considered configurations.

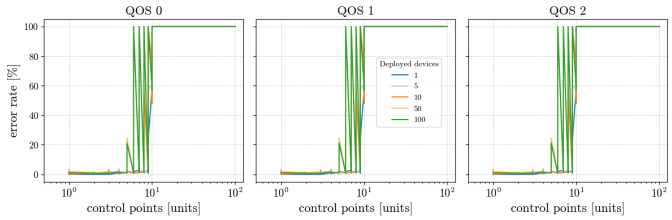


Fig. 5. MQTT robustness comparison with different *devices*, *control-points* and QOS. See 10a.

The Figure 5 shows a error rate comparison with different *devies*, *control-points* and QOS in MQTT-D. The obtained results show that the error rate rises sharply from almost 0% to 100% when the number of deployed *control-points* approaches 10 for every configuration. This suggests that, during the 10-second interval reserved for the discovery phase, none of the *control points* was able to receive the discovery announcement and respond within the required time frame.

1) Comparing the results: A comparison of the robustness of the two discovery protocols indicates that MQTT-D exhibits greater robustness when only a very small number of *control points* is deployed. However, once the threshold of 10 deployed *control points* is reached, MQTT-D displays an error rate of 100%, whereas UPnP still maintains a lower, although substantial, error rate corresponding to approximately 60% of *devices* not being discovered.

C. UPnP MX = 0

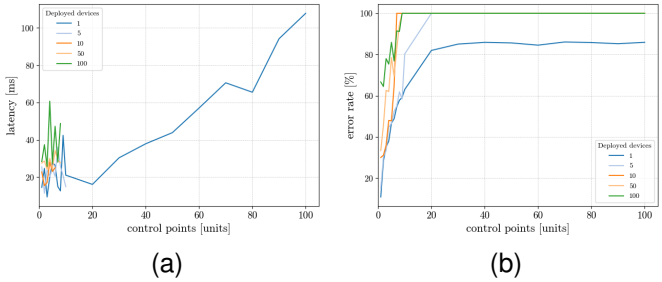


Fig. 6. UPnP (a) latency and (b) robustness different *devices control-points*. See 12a, 12b.

The experiment was performed by removing any artificial waiting time from the UPnP discovery process. Figure 6a shows the latency of UPnP when all the artificial waiting times are removed.

A comparison between Figure 6a and Figure 2 shows that the latency is reduced for all *device* configurations, with the most pronounced improvement occurring in scenarios with a smaller number of deployed *control points*.

In terms of robustness, Figure 6b indicates that, relative to Figure 4, the error rate rises more rapidly toward 100%. Nevertheless, the configuration with a single deployed *device* is able to complete the experiment without reaching a 100% error rate, thus exhibiting behaviour different from that shown in Figure 4.

D. GENA

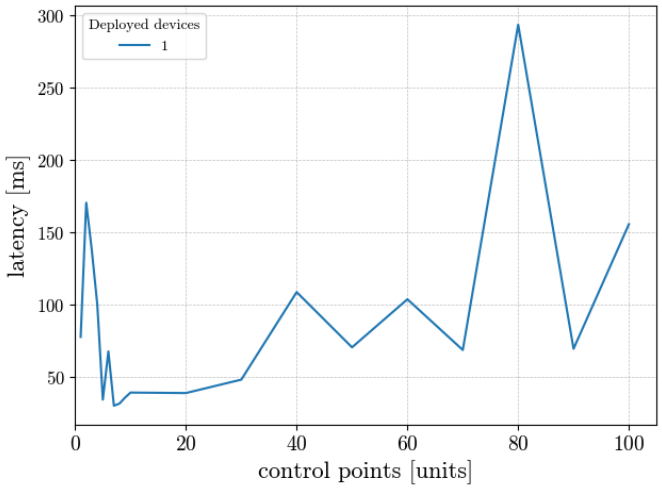
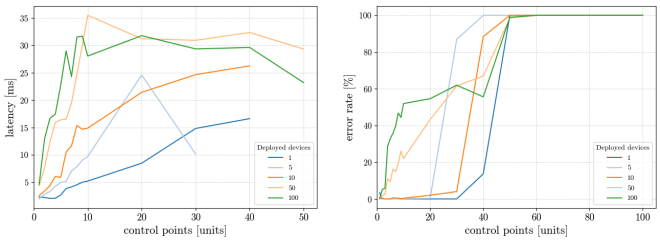


Fig. 7. GENA latency with different *control-points*.

The Figure 7 shows the delivery latency of unicast events as a function of the number of subscribed *control-points*. Data presented in Figure 7 indicates that for configurations with

fewer than 10 subscribers, latency and the number of *control-points* share an inverse relationship. Conversely, exceeding this limit results in a direct correlation, where additional *control-points* lead to increased latency.

E. Wireless vs Wired



(a) (b)

Fig. 8. Wired UPnP (a) latency and (b) robustness different *devices control-points*. See 11, 12b.

TODO

F. Discussion

The experiments proved UPnP to have less latency compared to MQTT-D. Configuration between 1 and 10 *control-points* deployed UPnP is from 5 to 10 times better depending on the deployed *devices*. For higher values of *devices* UPnP manages to remain almost constant, MQTT-D on the contrary goes to 0, that can be explained by looking the error rate that spikes to 100% meaning that not a single device is discovered.

Since in UPnP the RPC mechanism is implemented through SOAP that has a client-server approach, it is also interesting a comparison between GENA and MQTT-D both publisher-subscriber.

In the single-device deployment scenario, GENA exhibits the lowest latency among all QOS configurations evaluated within MQTT-D. Moreover, GENA consistently completes the experimental run, as its error rate never reaches 100%. Within the single-*device* setup featuring 1-4 control points, where a direct comparison between GENA and MQTT-D is feasible, GENA demonstrates substantially superior performance. Specifically, it achieves latency reductions of at least a factor of six in the single-*control-point* configuration and up to a factor of eight when two *control-points* are deployed.

Based on the evaluations conducted in this dissertation, UPnP demonstrated the lowest latency and the highest degree of robustness. These findings suggest that UPnP is better suited than MQTT-D for dense IoT deployments, particularly in scenarios where numerous devices must communicate concurrently in response to environmental changes that require timely reporting.

V. Conclusions

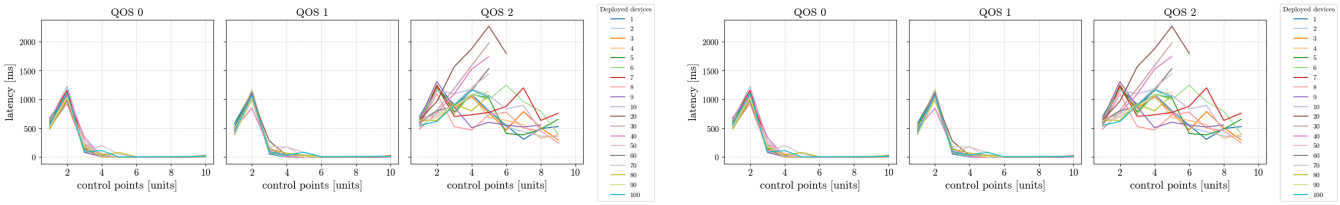
This dissertation undertook a comparative evaluation of MQTT-D and UPnP in dense network environments, with particular focus on latency and robustness under varying deployment configurations of *devices* and *control-points*. The experimental scenarios were assessed within a real network infrastructure specifically established for the purposes of this study. Each configuration was tested in both wired and wireless environments, the latter employing IEEE 802.11n technology.

For the purposes of these experiments, a fully compliant UPnP *device* and *control-point* were implemented. Nevertheless, to ensure compliance with the specifications, part of the *control-point* functionality relied on a third-party library.

The experimental results showed that UPnP achieved the lowest latency among all evaluated configurations. Furthermore, it demonstrated greater robustness, whereas MQTT-D completely failed to discover any devices at higher network densities. For MQTT-D, performance was strongly influenced by the QOS level adopted. In particular, the use of QOS 2 improved robustness, but at the cost of slightly higher latency, making it the least effective option in terms of response time.

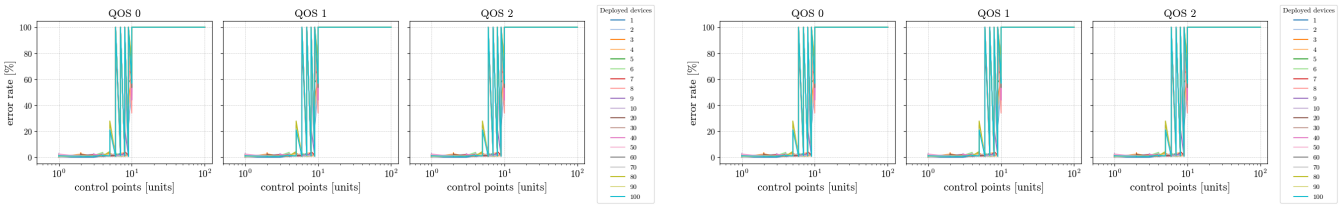
TODO mettere paragone wired wireless

Future research could investigate how UPnP and MQTT-D behave with real scenario data, investigating how payloads affect latency and robustness. In addition a further study could repeat this experiments in smart homes or industrial plant networks and evaluate how performances degrades in a shared network.



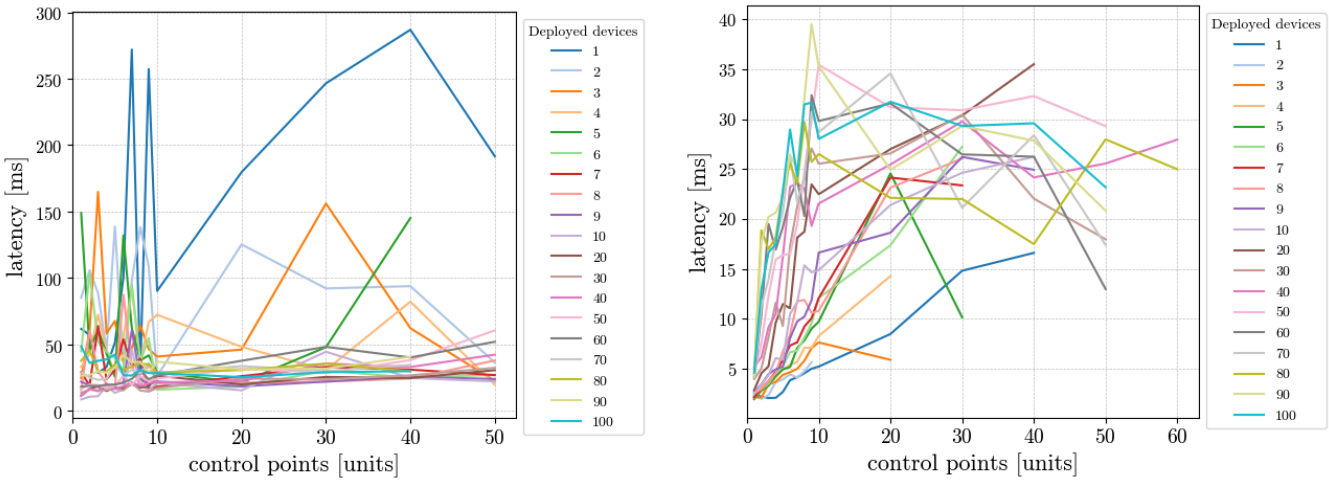
(a) (b)

Fig. 9. MQTT latency



(a) (b)

Fig. 10. MQTT latency



(a) (b)

Fig. 11. UPnP latency

Appendix

A. MQTT

1) MQTT latency:

2) MQTT robustness:

B. UPnP

1) UPnP latency:

2) UPnP robustness:

References

[1] I. T. Union, "Series y: Global information infrastructure, internet protocol aspects and next-generation networks," ITU-T, Recommendation Y.2060, February 2012. [Online]. Available: https://handle.itu.int/11.1002/1000/11559

[2] S. K. Datta, R. P. F. Da Costa, and C. Bonnet, "Resource discovery in internet of things: Current trends and future standardization aspects," in *2015 IEEE 2nd World Forum on Internet of Things (WF-IoT)*, 2015, pp. 542–547.

[3] A. Bröring, S. K. Datta, and C. Bonnet, "A categorization of discovery technologies for the internet of things," in *Proceedings of the 6th International Conference on the Internet of Things*, ser. IoT '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 131–139. [Online]. Available: https://doi.org/10.1145/2991561.2991570

[4] K. Khalil, K. Elgazzar, and M. Bayoumi, "A comparative analysis on resource discovery protocols for the internet of things," in *2018 IEEE Global Communications Conference (GLOBECOM)*, 2018, pp. 1–7.

[5] O. C. Foundation, "Upnp device architecture 2.0," Open Connectivity Foundation, Tech. Rep., April 2020, document Revision Date: April 17, 2020. [Online]. Available: https://openconnectivity.org/upnp-specs/UPnP-arch-DeviceArchitecture-v2.0-20200417.pdf

[6] T. Berners-Lee, R. Cailliau, A. Luotonen, H. F. Nielsen, and A. Secret, "The world-wide web," *Commun. ACM*, vol. 37, no. 8, p. 76–82, Aug. 1994. [Online]. Available: https://doi.org/10.1145/179606.179671

[7] M. Achir, A. Abdelli, and L. Mokdad, "A taxonomy of service discovery

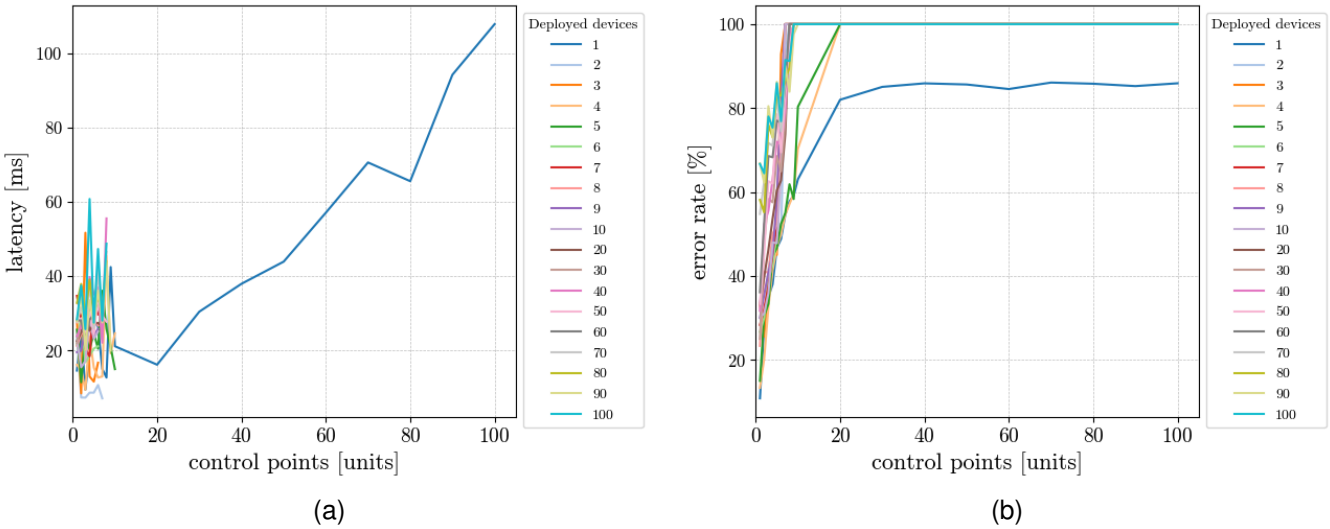


Fig. 12. UPnP (a) latency and (b) robustness with no random delay

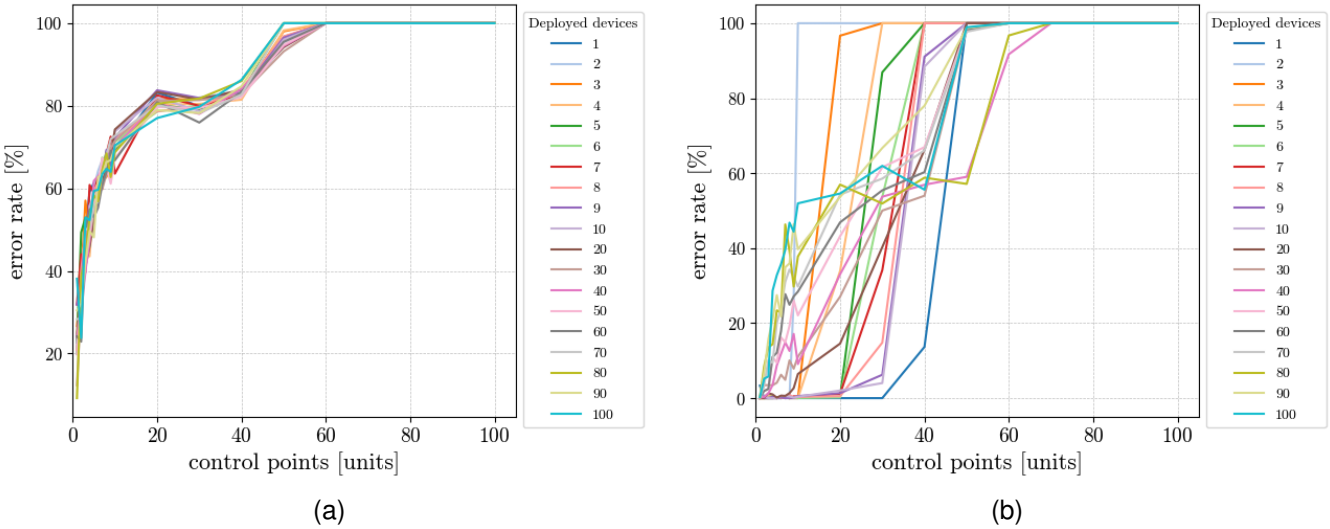


Fig. 13. UPnP robustness

approaches in iot," in *2020 8th International Conference on Wireless Networks and Mobile Communications (WINCOM)*, 2020, pp. 1–6.

[8] DaniDF, "MQTT-Discovery vs UPnP," https://github.com/DaniDF/MQTT-Discovery-vs-UPnP, 2026.

[9] Huin, "goupnp," https://github.com/huin/goupnp, 2026.

[10] O. H. Foundation, "Mqtt discovery," https://www.home-assistant.io/integrations/mqtt/#mqtt-discovery, 2026.

[11] I. C. Society, "Ieee standard for information technology– local and metropolitan area networks– specific requirements– part 11: Wireless lan medium access control (mac)and physical layer (phy) specifications amendment 5: Enhancements for higher throughput," *IEEE Std 802.11n-2009 (Amendment to IEEE Std 802.11-2007 as amended by IEEE Std 802.11k-2008, IEEE Std 802.11r-2008, IEEE Std 802.11y-2008, and IEEE Std 802.11w-2009)*, pp. 1–565, 2009.